

TrainAudit: Silent Error Detection for LLM Training via Verified Guarded Relational Constraints—Supplementary Material

Anonymous Author(s)

Abstract

This PDF is *review-only* supplementary material for the main paper. The main paper is self-contained: reviewers are not required to read this file (SIGMOD/PACMMOD appendix policy). Content here is limited to methodology details, full pattern/SQL/algorithm specifications, and per-case tables that support claims already stated in the main text. Sections marked *implementation detail* are for reproducibility and may be skipped when checking scientific claims. This appendix is not part of the camera-ready.

Roadmap: Appendix Sections vs. Main Paper

Table .1 maps each appendix section to the main-paper claim it supports. Primary reading order for verifying main claims: B→D→G→I→O→L/M.

Table .1: Where to look when checking a main-paper claim. Labels match those cited from the main text.

App.	Supports (main)	Content
A	§Related Work	Extended oracle/scope comparison table
B	§Empirical / Real-SE setup	Pool construction, silent filter, IRR, reproducibility
C	Table 1 taxonomy	Fine-grained 392-case distribution figure
D	§Pattern Catalog	Frozen induction protocol, reliability, and held-out coverage
E	§Pattern Catalog	Constraint-taxonomy overview (no new claims)
F	§Online Detection	End-to-end SwitchMLP running example
G	§Invariant Miner	FSM, funnel algorithm, I/O records, provenance audit
H	§Invariant Miner	Abridged LLM prompt templates (implementation)
I	§Online Detection / theory	Trace schema and compiled SQL (incl. group semantics)
J–K	§Data Collector / Efficiency	Collector hooks and batch runner (implementation)
L	Diagnosis paragraph	C1/C2 drill-down measurements on detected cases
M	Out-of-pool / production	Three production case studies
N	Motivation figures	Figures moved out of the main paper for space
O	§Detection / Efficiency	Per-case table, tier curve, hookpoint list

A Extended Related Work Comparison

This appendix expands the related-work comparison in the main text along oracle source, deployment mode, and scope of the checked property. The data-quality row mirrors the main-text positioning of Auto-Validate / Deequ / denial constraints relative to topology-conditioned runtime guards.

B Bug Pool Construction and Methodology

This appendix documents the source, filtering, classification, and inter-rater validation of the 392-record silent-error evidence corpus. The merged manifest, deterministic mapping tables, disagreement sets, provenance audit, and the available reproduction artifacts are released with the paper.

B.1 Pool Construction

Our evidence corpus covers four open-source distributed training frameworks: Megatron-LM, DeepSpeed, OLMo, and OLMo-core. We selected them because each implements training-critical distributed logic directly and exposes auditable source history. The corpus frozen on 2026-05-10 merges two earlier collection channels: a 295-record pool with commit/reproduction-oriented metadata and a 128-record issue/PR-curated pool. Thirty-two incidents overlap; after retaining the 96 records unique to the earlier pool, all 295 records from the commit-oriented pool, and one fully specified orphan, the merged artifact contains **392 records**. A second incomplete orphan outside both source pools was considered and excluded before assembly. The final framework counts are: Megatron-LM 110, DeepSpeed 123, OLMo 77, and OLMo-core 82.

Why commits are preferred to issue-first search. Issue and PR text is useful for discovering and interpreting a defect, but is not a sufficient experimental unit. Reports frequently omit the exact buggy revision, triggering configuration, executable oracle, or fixed comparison; some combine several defects or describe only a downstream symptom. A fix commit instead supplies a concrete code transition, and its parent or named pre-fix revision supplies a candidate buggy state. We therefore treat commit diffs/messages as the preferred semantic evidence and use issue/PR text only as supporting context. The released merged corpus nevertheless retains 96 records from the earlier issue/PR-only channel so that the catalog study does not discard potentially recurring relations merely because a replay is unavailable. These weaker records never enter Real-SE without a later commit-pair audit.

Screening and deduplication. A candidate remains only when (i) it affects a training-correctness path such as communication, parallelism, optimizer state, checkpointing, numerical computation, or data loading rather than documentation, performance, or refactoring alone; (ii) the evidence states a semantic violation—for example divergence, wrong scaling, incomplete restoration, or an incorrect call count; (iii) the buggy behavior can continue without an immediate exception as its primary manifestation; and (iv) the changed code or curated source summary is specific enough to extract semantic objects, an expected relation, and a possible witness. Records referring to the same upstream incident are collapsed before analysis.

Provenance strength and collection limitation. The merged manifest contains 289 records with a fixed-commit identifier, 96 with an issue/PR/release URL but no fixed commit, and 7 whose merged record retains only a curated source summary. Of the 289 fixed identifiers, 288 resolve in the released local Git objects; their dates span 2021-08-06 through 2026-05-06. This is the observed span of retained fixes, not a predefined crawl interval. The current artifact does not preserve a raw ledger of every commit inspected, repository snapshot SHAs at a uniform crawl boundary, or per-candidate exclusion counts. We therefore do *not* characterize the 392 records as an exhaustive scan of every default-branch commit. The exact audit is released as `benchmark/eval/corpus_provenance_audit.md`; reconstructing a strict all-commit census would require rebuilding the corpus and rerunning the frozen template-induction split.

B.2 Silent-vs-Loud Filtering

The main text states three defining criteria for a silent error—no immediate crash, surface-plausible metrics, and semantic impact. These cannot all be physically replayed for a large survey corpus. We therefore separate evidence strength rather than treating every record as equally confirmed: keyword and metadata ratings triage records, while only runnable buggy/fixed comparisons provide physical evidence.

4-tier evidence rating. We assign each bug an evidence tier (Table B.2). **L1 (n=23)** is the strongest evidence: the bug is reproduced, the buggy run completes without raising an exception, and the recorded `expected_output` field differs from the measured fixed-run output—this is silent in the physical sense. **L2 (n=161)** is medium evidence: the `root_cause` description contains silent keywords (*silently*, *accumulate*, *drift*, *diverge*, *mismatch*) and no crash keywords (*raise*, *assert*, *NaN*, *Inf*, *OOM*, *Exception*). **L3 (n=72)** is weaker evidence: only the issue title carries silent signals. **L4 (n=136)** is uncertain: the fields are sparse—short issue description or a fix diff spanning multiple files that resists attribution—retained but routed into a manual review pool. The rating script `silent_evidence_rate.py` writes per-bug tiers and triggering keywords into `silent_evidence_392.json`, allowing line-by-line audit.

Reverse loud-bug scan. We use the absence of a newly added `raise` or `assert` only as supporting counter-evidence, not as proof that a case is silent. A reverse scan reports no such addition among the available fix diffs, while a crash-keyword scan produces two warnings that are retained after manual adjudication: D-037 combines one topology-specific crash with two silent accumulation defects, and OC-NEW-21 reports either a crash or a silent overwrite depending on the path. Per-record evidence tiers and warnings are released in `silent_evidence_392.json` and `silent_evidence_warnings.csv`.

Physical anchors. Physical evidence comes from the runnable subsets rather than from adding the textual tiers together: the 23 L1 reproductions, the audited Real-SE commit pairs, and the severe/medium software fault replays. These sets serve different experiments and may overlap. For each included runnable instance, the archived verdict compares buggy and fixed behavior directly; this is the evidence that keyword or metadata screening cannot replace.

B.3 Descriptive Subsystem Coding

For descriptive corpus reporting, each record receives one coarse primary subsystem label. The label vocabulary is not defined a priori: we performed open coding on a deeply annotated subset—freely assigning 5–10 semantic tags per record—followed by axial clustering along framework subsystem boundaries. This process yielded 13 reporting labels: *numerical*, *control_flow*, *gradient_sync*, *checkpoint*, *moe*, *optimizer_state*, *data_loading*, *communication*, *dtype*, *loss_computation*, *sharding*, *lr_schedule*, and *offload*. These are coarse descriptive bins, not a unique or exhaustive fault ontology and not the Pattern Catalog’s relation-template space. Definitions and per-record attribution rationales are released under `taxonomy_methodology.md`.

A subset of records in the pool ($n = 295$) carry initial labels outside the final reporting vocabulary, inherited from an earlier annotation schema, and require alignment. Alignment proceeds in three steps: **138/295 (46.8%) match directly**; **154/295 (52.2%)** are mechanically mapped via a 60-entry deterministic ALIAS_MAP (e.g., `"weight_init"` → *numerical*, `"async_op"` → *communication*); and the remaining **3/295 (1.0%)** are independently judged by two annotators and adjudicated when no rule matches. The released ALIAS_MAP and `build_392_catalog.py` reproduce this descriptive coding output.

B.4 Inter-Rater Reliability

Coding consistency cannot rest on a single metric. We adopt a dual-protocol cross validation: protocol A measures re-annotation agreement using 32 doubly-and-independently annotated records in the pool; protocol B introduces an LLM rater fully independent of the original annotators and computes Cohen’s κ on a 50-record stratified sample.

Protocol A: Re-annotation agreement. 32 records in the pool went through two mutually independent annotation flows, forming a natural inter-rater test set. We observe $25/32 = 78.1\%$ **raw agreement**; all 7 disagreements are released with per-record rationale, and the disagreements concentrate between the broad *control_flow* label and more specific subsystem labels (*lr_schedule*, *gradient_sync*).

Protocol B: Independent LLM rater. We drew a 50-record sample stratified over the reporting labels, hid the reference labels, and had an independent LLM rater assign labels based only on the issue title, `root_cause`, and fix diff. Raw agreement is 60.0% (30/50); under the chance baseline of $p_{\text{expected}} = 0.078$ for the observed label vocabulary, this gives **Cohen’s $\kappa = 0.566$** . This moderate agreement reinforces why the labels are used only for coarse corpus description rather than as the detector’s semantic representation. `irr_50_results.json` releases the stripped input, independent and reference labels, per-label metrics, and all disagreement details. Table B.3 summarizes the two protocols.

Literature benchmark. $\kappa = 0.566$ is comparable to reported multi-class systems-bug coding studies. DeepLearningBench (ICSE'20) reports $\kappa = 0.59$ on an 8-label DL bug taxonomy; PROMISE-series systems-bug studies commonly report κ in the 0.55–0.75 range.

B.5 Coding-Label Boundary Limitations

The coarse reporting labels have ambiguous boundaries; we report the two systematic confusion patterns surfaced by per-label F1 in the IRR analysis.

The control_flow label is broad. All 4 reference *control_flow* records are reassigned by the protocol-B rerater to more specific subsystem labels (*lr_schedule*, *gradient_sync*, *optimizer_state*), driving this label's per-label F1 to 0. We retain it because some records describe a control-flow root cause that is not specific to another subsystem.

gradient_sync over-prediction. In the reverse direction, *gradient_sync* is over-predicted in protocol B (reference=4, LLM=8): the rerater moves records from *numerical* and *data_loading* that exhibit a gradient-related symptom into *gradient_sync*—a typical bias of coding by *symptom* rather than by *root cause*.

Several labels are more stable. By contrast, 5 labels reach $F1 \geq 0.80$ in protocol B: *loss_computation* (1.00), *lr_schedule* (0.86), *checkpoint* (0.86), *dtype* (0.80), and *sharding* (0.80).

B.6 From the Evidence Corpus to Real-SE

Real-SE is not obtained by treating the incomplete *reproduction_status* field as a random sample. We first use the descriptive subsystem labels and four frameworks as diversity targets, then require every admitted detector-coverable case to satisfy six criteria: (i) auditable upstream provenance; (ii) a silent semantic failure rather than a synthetic fault; (iii) an explicit buggy/fixed commit comparison; (iv) a runnable driver that triggers the ground-truth violation on the buggy revision and remains clean on the fixed revision, or observes the fixed revision reject the invalid configuration; (v) archived commands, environment, seed/GPU count, logs, and verdict; and (vi) no pattern-only surrogate substituted for the upstream bug. We freeze both these criteria and the resulting manifest before detector scoring, so case admission and exclusion are independent of detection outcomes.

Applying this filter yields 17 detector-coverable native-commit method-level replays, with at least one representative from each descriptive subsystem label, and one real observability-boundary case, O-003. O-003 is deliberately counted as a miss because its sub-percent signal lies below the current online schema. O-022 is excluded from Real-SE because it lacks a reproducible commit pair, although it remains evidence for the survey and catalog. Real-SE is therefore a coverage-constrained executable benchmark, not a random sample of the 392 records; it favors bugs with localizable method-level witnesses and its detection rate is not a prevalence estimate for the full corpus. The manifest, selection reasons, demotions, and per-case provenance are released under `benchmark/eval/real_sdc/`.

Each detector-coverable entry is a *native-commit method-level replay*: its driver executes the affected methods at their upstream buggy and fixed revisions using constructed inputs and short-circuited collectives. This preserves the upstream fault semantics in a compact executable witness, but it does not claim an end-to-end training reproduction. The separately designated boundary case is counted in the buggy-case denominator but does not require a paired fixed-side evaluation.

Pool assembly and descriptive-coding artifacts can be regenerated with:

```
python3 benchmark/eval/build_392_catalog.py
python3 benchmark/eval/build_manifest_v2.py
```

Expected outputs: 392/392 records assigned one of the 13 descriptive labels; 7/32 re-annotation disagreements; IRR $\kappa = 0.566$.

C Fine-Grained Distribution of the 392-Record Evidence Corpus

The main text gives a compact distribution over the coarse primary subsystem labels. This appendix presents the finer distribution—both by reporting label and by training parallelism dimension—to facilitate experimental reproduction and inspect sampling diversity.

D Template Induction Protocol and Results

This appendix documents the frozen protocol used to construct and validate one versioned snapshot of the Pattern Catalog. The deployed catalog is designed to evolve with framework code and newly confirmed bug evidence; freezing here is an experimental control, not a claim that the catalog has a permanent size. Before any annotator saw the locked test, we fixed the dataset split, extraction fields, merge/split rules, recurring-template admission rule, and stopping criterion. The 392 records are divided into 128 seed records, 186 development records, and 78 locked-test records; records from the same upstream incident remain in one split, and all prior template labels and invariant fields are hidden from annotators.

Extraction and template boundaries. For each case, annotators record the framework-independent semantic objects, expected relation, observable witness, topology scope, training phase, and validity precondition. Two cases merge only when their relation, object roles, and witness agree and all remaining differences can be expressed as guards. They split when a merge changes the relation, requires unrelated disjunctions, lacks a common observable signal, or flags a legal execution. A template enters the recurring catalog only after two independent incidents support it; one-off relations remain singletons.

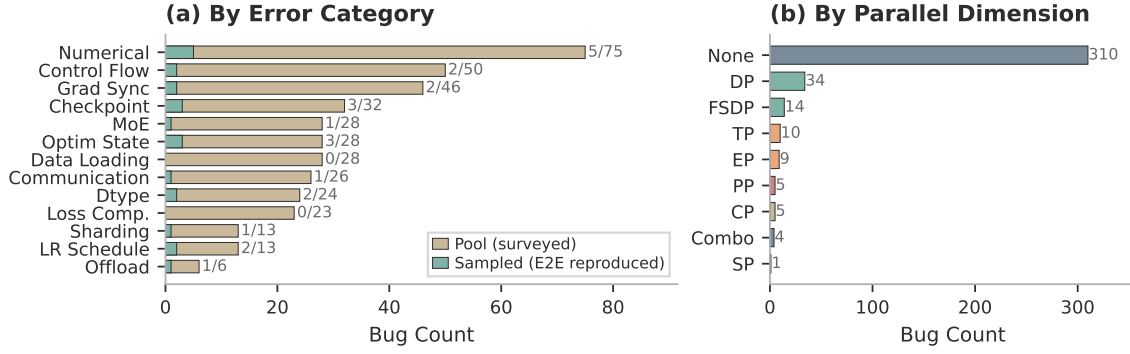


Figure C.1: Fine-grained distribution of the 392-record evidence corpus. (a) By descriptive subsystem label: the background color (sand) marks the corpus, while the foreground (teal) marks historical pre-filter diversity targets used during Real-SE construction; s/p denotes target/pool. (b) By dominant training parallelism dimension (None = no obvious dimension dependency; Combo = multi-dimension composite).

Reliability and generalization. Open relation extraction is the least stable step: double-coding 79 construction cases gives $\kappa=0.600$ on the relation operator, largely because annotators choose different names for the same obligation. Once the catalog is frozen, independent assignment becomes highly reproducible: template assignment reaches $\kappa=0.954$, match-versus-uncovered reaches $\kappa=0.843$, and the six-way verdict reaches $\kappa=0.772$. On the locked test, schema coverage is 72/78 (92.3%), joint grounding is 71/78 (91.0%), and the novelty rate is 3/78 (3.8%); none of the three novel relations recurs twice.

Snapshot growth and scope of the claim. The preregistered stopping condition never fires: this experimental snapshot grows from 27 to 35 recurring templates and the final development batch still adds two. Its canonical relation vocabulary grows from 18 to 20, no accepted template requires a core-relation revision, and coverage stabilizes near 0.86 early in the development stream. These numbers characterize only the corpus and catalog version frozen for this study. We therefore claim that relation templates are reusable while the catalog remains extensible as code and evidence evolve—not that the system has a complete, globally minimal, saturated, or fixed-size catalog. A pairwise audit finds no mergeable pair among the snapshot’s 35 templates under the frozen criterion, but this establishes only criterion-relative irredundancy. The full protocol, catalog definitions, annotations, disagreements, and order-robustness traces are released under `benchmark/eval/template_induction/`.

E Detailed Constraint Taxonomy

This appendix presents the detailed taxonomy of the invariants discovered by our system, organized by semantic scope and verification granularity ranging from model-level consistency down to tensor-level numerical correctness.

F Case Study: End-to-End Error Detection Workflow

This appendix provides a detailed walkthrough of the TRAINAUDIT detection workflow using Case B10 (`micro_step_id` inconsistency in ZeRO-2 with CPU offload) as a representative example.

F.1 Problem Background

The DeepSpeed ZeRO-2 optimizer with CPU offloading contains a subtle bug in gradient accumulation logic. The `micro_step_id` counter exhibits inconsistent behavior between initialization and reset phases.

F.2 Detection SQL Query

The following SQL query was automatically generated to detect the `micro_step_id` inconsistency:

Detection SQL Query

```

1 WITH step_counter_analysis AS (
2   SELECT
3     step,
4     json_extract_string(data, '$.micro_step_id') as micro_step_id,
5     json_extract_string(data, '$.gradient_op') as gradient_op,
6     json_extract_string(data, '$.expected_op') as expected_op

```

```

7   FROM coredump
8   WHERE stage = 'gradient-accumulation'
9 )
10  SELECT step, micro_step_id, gradient_op, expected_op
11  FROM step_counter_analysis
12  WHERE gradient_op != expected_op
13  ORDER BY step;

```

F.3 Detection Result

The query returned 12 anomalous records where the gradient operation was “Copy” when “Accumulate” was expected, confirming the initialization-reset inconsistency.

G Invariant Miner: Complete Algorithm Specification

This appendix provides the complete algorithmic specification of the Invariant Miner.

G.1 Finite State Machine Formalization

Definition G.1 (Invariant Miner FSM). The Invariant Miner is formalized as a 5-tuple $\mathcal{F} = (S, A, \delta, s_0, F)$ with states $S = \{S_1, \dots, S_5\}$ (cf. Table G.1), agent set $A = \{\text{GAP-ANALYSISAGENT}, \text{EVIDENCEAGENT}, \text{SYNTHESISAGENT}, \text{PERSISTENCEAGENT}, \text{REPORTINGAGENT}\}$, transition function $\delta: S \times \Sigma \rightarrow S$, initial state $s_0 = S_1$, and accepting state $F = \{S_5\}$.

The FSM formalization captures the high-level state transitions, while the concrete verification logic is specified algorithmically below.

G.2 Four-Stage Verification Funnel

Algorithm 1 presents the complete procedure for mining a single constraint with bidirectional verification. The algorithm takes as input the existing constraint library $\mathcal{L}$ and the current training context C , and outputs either a validated constraint c^* or a rejection signal $\perp$.

Per-stage candidate counts and skip-stress validation. Figure G.1 gives the per-stage counts across four target frameworks. The 420 pattern hypotheses at Stage 1 expand to 5,334 concrete candidates at Stage 2; clean-trace validation retains 3,436 at Stage 3; adversarial verification retains 357 at Stage 4; and a separate manual cross-framework curation selects 45 deployment rules. Without Stage 3, 114/400 sampled candidate–trace evaluations fire on held-out clean traces (28.5%). In a separate held-out check, full-funnel survivors raise 0 alerts in 764 evaluations. Stage 4 further removes workload-specific candidates that same-workload clean traces do not falsify.

Table G.2: Leave-one-database-out guard ablation over 42 evaluation databases. ΣFP aggregates false-positive query outcomes. Every database preserves the ordering $\text{full} < \neg\pi_{\text{precond}} < \text{no adversarial verification} < \neg\pi_{\text{topo}}$.

Library variant	ΣFP	vs. full
Full	342	—
$\neg\pi_{\text{precond}}$	429	+25.4%
$\neg$ adversarial verification	551	+61.1%
$\neg\pi_{\text{topo}}$	598	+74.9%

Why the candidate space must be searched, not enumerated by hand. Figure G.2 illustrates why verification, rather than manual curation, is the binding constraint on mining. A single trace relation generalizes along four independent slots—the relation template, the parameter scope, the topology guard π_{topo} , and the phase guard π_{precond} —and every combination of choices is a candidate constraint that an LLM could plausibly propose from framework evidence. Across the 4 target frameworks this enumeration produces the 5,334 concrete candidates of Figure G.1; most encode workload-specific coincidences, and only 357 survive counterexample verification.

G.3 State Transition Rules

Table G.3 specifies the complete state transition rules with routing keywords.

G.4 Concrete Input/Output Specifications

This section provides concrete data formats for each stage of the four-stage verification funnel.

Stage 1: Gap Analysis Output.

```

{
  "constraint_hypothesis": "DP optimizer_state consistency",
  "gap_identified": "optimizer internal states (momentum,
    variance) not verified across DP ranks",

```

Algorithm 1 Invariant Mining with Bidirectional Verification**Require:** Constraint library $\mathcal{L}$, Training context C **Ensure:** Validated constraint c^* or $\perp$ (reject)

```

1: // Stage 1: Gap Analysis
2:  $\mathcal{G} \leftarrow \text{ANALYZEGAPS}(\mathcal{L}, C)$ 
3:  $c_{hyp} \leftarrow \text{PROPOSEHYPOTHESIS}(\mathcal{G})$ 
4: if  $\text{IsSEMANTICEQUIVALENT}(c_{hyp}, \mathcal{L})$  then
5:   return  $\perp$  {reject duplicate}
6: end if
7: // Stage 2: External Research
8:  $\mathcal{E} \leftarrow \text{AGGREGATIONEXPERT.SEARCH}(c_{hyp})$ 
9: if  $|\mathcal{E}.\text{sources}| < 2$  then
10:  return  $\perp$  {insufficient evidence}
11: end if
12:  $c_{ref} \leftarrow \text{REFINECONDITIONS}(c_{hyp}, \mathcal{E})$ 
13: // Stage 3: Counterexample Construction
14:  $C\mathcal{E} \leftarrow \emptyset$ 
15: for  $i = 1$  to 2 do
16:    $ce_i \leftarrow \text{CONSTRUCTCOUNTEREXAMPLE}(c_{ref})$ 
17:    $r_i \leftarrow \text{VERIFY}(ce_i)$ 
18:    $C\mathcal{E} \leftarrow C\mathcal{E} \cup \{(ce_i, r_i)\}$ 
19: end for
20: if  $\exists (ce, r) \in C\mathcal{E} : r = \text{HOLDS}$  then
21:  return  $\perp$  {counterexample valid}
22: end if
23: // Stage 4: Final Decision
24:  $conf \leftarrow \text{EVALUATECONFIDENCE}(c_{ref}, \mathcal{E}, C\mathcal{E})$ 
25: if  $conf \geq 0.8$  then
26:   $\text{PERSIST}(c_{ref})$ 
27:  return  $c_{ref}$ 
28: else
29:  return  $\perp$  {low confidence}
30: end if

```

```

"initial_conditions": ["dp > 1",
  "stage = model-after-backward"],
"preliminary_reasoning": [
  "Adam/LAMB maintain internal buffers",
  "Buffers must sync for deterministic training"
]
}

```

Stage 2: External Research Output.

```

{
  "research_query": "Megatron-LM optimizer state DP sync",
  "sources_found": 3,
  "evidence": [
    {
      "source_type": "source_code",
      "path": "megatron/optimizer/distrib_optimizer.py",
      "relevance": 0.95,
      "excerpt": "all_reduce optimizer states across DP group"
    },
    {
      "source_type": "documentation",
      "path": "docs/distrib_optimizer.md",
      "relevance": 0.88,
      "excerpt": "DP ranks maintain synchronized optimizer states"
    }
  ]
}

```

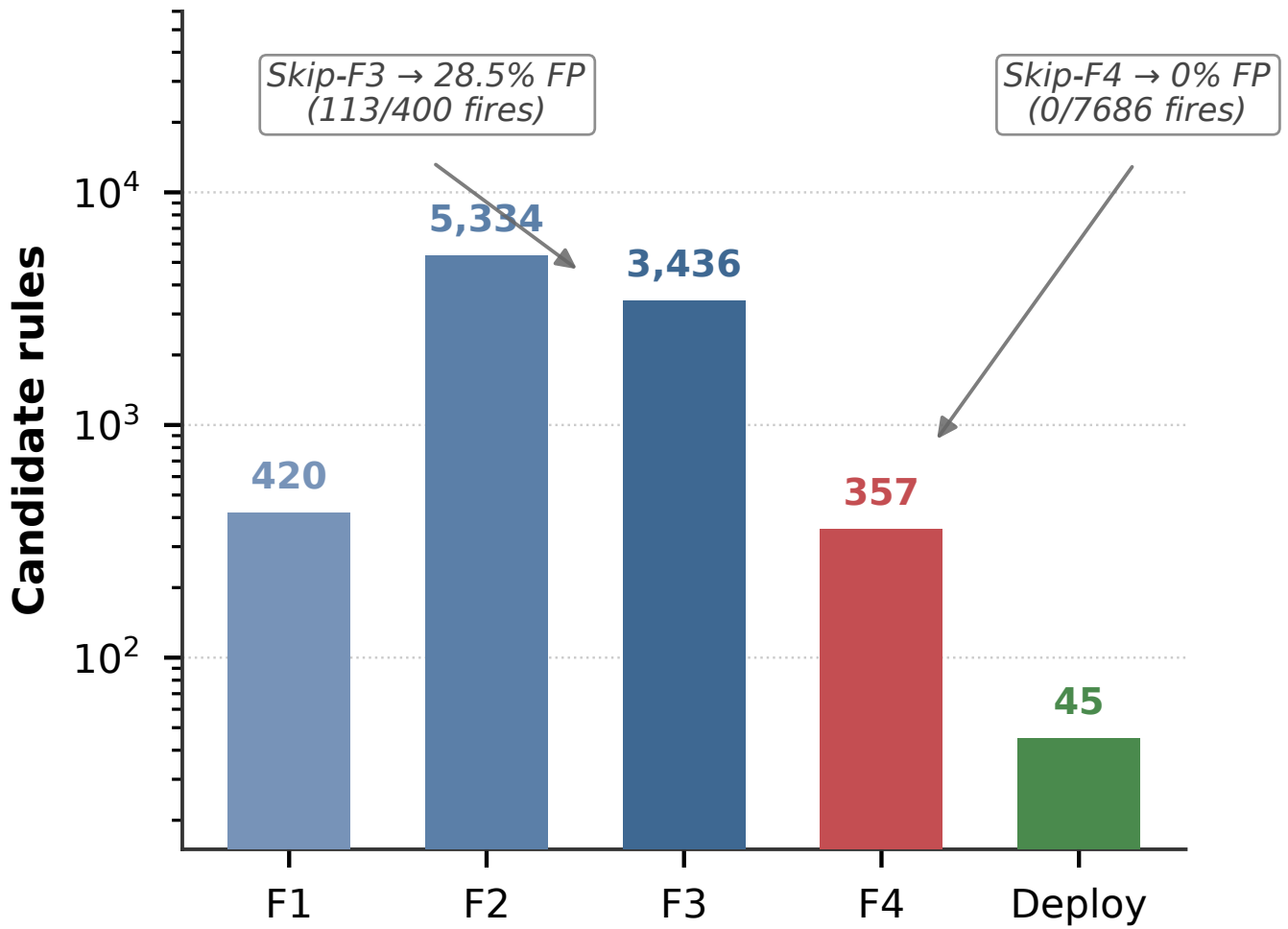


Figure G.1: Per-stage candidate counts of the four-stage mining funnel (aggregated across 4 frameworks). F1–F4 correspond to Stages 1–4 of Algorithm 1; Deploy is the deployment library after manual cross-framework curation. *Note:* F1–F4 here denote funnel levels and are a different axis from the evidence tiers L1–L4 of §B.2.

```

],
"sufficient_evidence": true
}

Stage 3: Counterexample Verification Record.
{
  "counterexample_id": 1,
  "scenario": "What if using SGD without momentum?",
  "query": "Megatron-LM SGD optimizer state sync",
  "verification_result": "REFUTED",
  "evidence": "SGD has no internal state to sync",
  "source": "megatron/optimizer/optimizer.py:L142-L156"
}

Stage 4: Final Constraint Output.
{
  "name": "DP optimizer_state consistency after backward",
  "description": "Optimizer internal states must be

```

One trace relation generalizes along four slots:

every combination is a candidate constraint the LLM could plausibly propose.

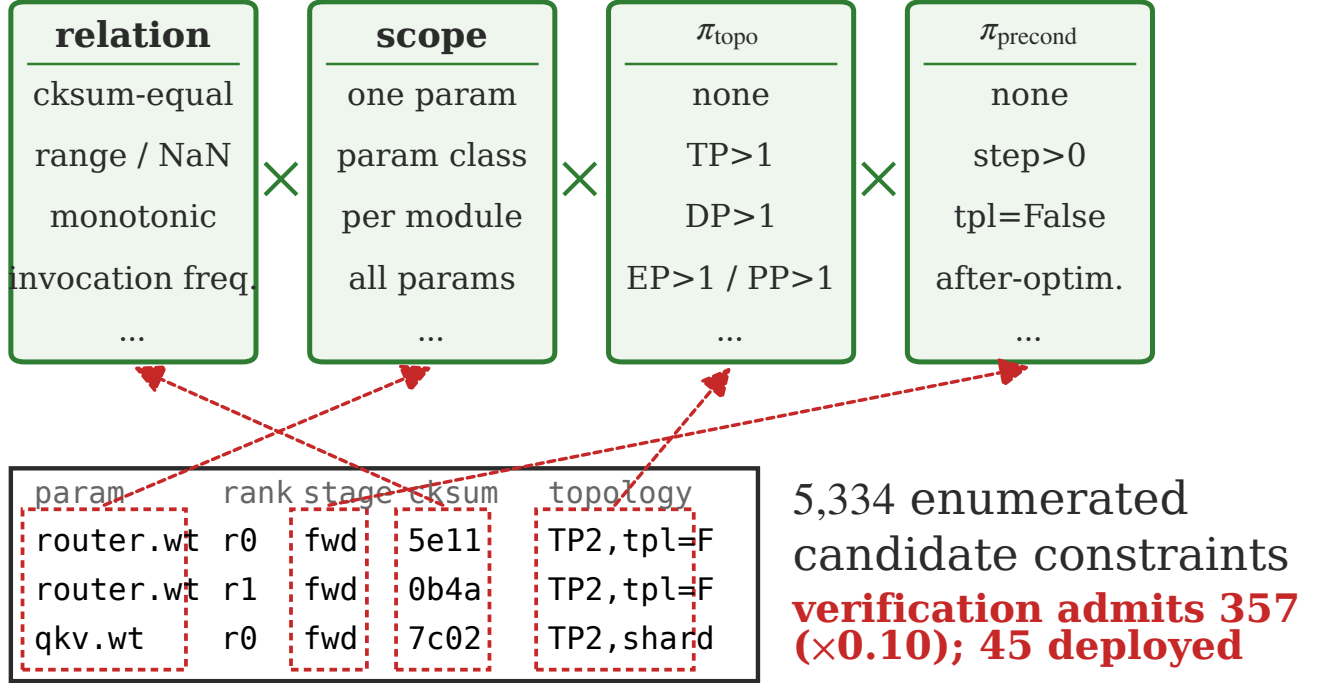


Figure G.2: Candidate-space explosion. Each slot of a trace relation admits multiple generalizations (dashed boxes trace slots back to raw trace columns); the cross product yields 5,334 enumerated candidates, of which verification admits 357 ($\times 0.10$) and 45 are deployed.

```

    identical across all DP ranks",
    "type": "consistency",
    "applicable_conditions": {
      "dp": "> 1",
      "stage": "= 'model-after-backward'",
      "optimizer_type": "IN ('Adam', 'LAMB')",
    },
    "confidence": 0.92,
    "evidence_count": 3,
    "counterexamples_tested": 2,
    "counterexamples_refuted": 2
  }

```

G.5 Mining Provenance Audit Protocol

This subsection records the protocol behind the main-text *mining provenance* claim (Generalizability): case-level Real-SE material does not enter the mining inputs.

- **Pre-registered exclusion list.** The 18 Real-SE case IDs, frameworks, and (when available) fix-commit short hashes are pre-registered in a JSON exclusion list shipped with the artifact; the list supports the audit below and any future from-scratch re-mining study. It is derived from Table O.1 and is not edited after registration.
- **Primary library audit.** The archived mining-session logs used to build the deployed Megatron-LM rule library (150 `llm_interactions_*.log` files covering the construction window) were scanned for (i) every fix-commit hash on the exclusion list and (ii) any GitHub issue/PR

URL of the form `github.com/.../(issues|pull)/[0-9]+`. Both scans returned zero hits: the miner messages contain framework source paths, documentation excerpts, and injection-lineage tables, not case-level bug reports.

- **What this does and does not show.** Zero log hits rule out case-level leakage into the recorded mining transcripts. They do not rule out LLM pretraining exposure to public bug discussions—that residual channel is stated in the main-text Limitations paragraph.

H LLM Prompt Templates

Implementation detail. Reviewers verifying main-paper claims may skip this section. Prompt text below is abridged for reproducibility of the offline miner; the scientific contribution is the verified mining loop (Appendix G), not the wording of any single prompt.

This appendix documents the abridged prompt templates used in the Invariant Miner.

H.1 Prompt Design Principles

Our prompt design follows three key principles: **(1) Role Specification**—each agent receives a clear role definition with explicit responsibilities and boundaries; **(2) Structured Output**—prompts specify JSON output schemas to ensure parseability; **(3) Constraint Enforcement**—routing rules and forbidden actions are explicitly enumerated.

H.2 Agent Prompt Summary

H.3 Training Expert Prompt (Abridged)

Training Expert System Prompt (Abridged)

Role: You are `megatron_expert` (adversarial verification expert).

Core Task: Generate high-quality constraints through 4-stage funnel filtering.

Stage 1: Initial Screening (Forward Hypothesis)

- Call `analyze_existing_constraints()` to get current library
- Identify uncovered gaps (stage/parameter/parallel strategy)
- Propose candidate constraint hypothesis
- **Output:** `NEXT_ACTION: CONTINUE_AGGREGATION`

Stage 2: External Evidence Collection

- Request `AggregationExpert` to gather external evidence
- Refine conditions based on documentation
- **Required:** ≥ 2 independent sources

Stage 3: Counterexample Verification

- Construct ≥ 2 adversarial scenarios
- Request verification via `VERIFY_COUNTEREXAMPLE`
- Evaluate counterexample validity

Stage 4: Final Decision

- Output `WRITE_CONSTRAINT` only if all stages pass
- Include: `UNIQUENESS_CONFIRMED`, `EXTERNAL_EVIDENCE`, `COUNTEREXAMPLE_RESULTS`

Forbidden Actions:

- × Output `WRITE_CONSTRAINT` before Stage 4
- × Skip external research phase
- × Self-judge counterexamples without verification

H.4 Aggregation Expert Prompt (Abridged)

Aggregation Expert System Prompt (Abridged)

Role: You are aggregation_expert (Web Researcher with strict citation).

Core Principle: Absolute authenticity - every piece of information must be traceable.

5-Layer Verification:

- (1) **Source Verification:** Only official docs, papers, code, issues
- (2) **Timeliness:** Must include version and date
- (3) **Cross-Validation:** ≥ 2 independent sources per claim
- (4) **Citation Completeness:** URL + version + quote required
- (5) **Verifiability:** File path + line number for code

Output: Call submit_research() immediately after completion.

Boundary: Research only — no decision making, no constraint generation.

H.5 Coordinate Agent Routing Rules

Coordinate Agent Routing Table

Message Feature

CONTINUE_AGGREGATION
 VERIFY_COUNTEREXAMPLE
 research_submitted=True
 WRITE_CONSTRAINT + evidence
 GENERATE_REPORT

Lineage Enforcement:

- × Never route directly from S_1 to WRITE_CONSTRAINT
- × Never skip megatron decision after aggregation
- × Never allow < 2 counterexamples before Stage 4

Route To

aggregation
 aggregation
 megatron
 write_agent
 report_agent

Function Call

go_aggregation()
 go_aggregation()
 go_megatron()
 go_write_agent()
 go_report()

I SQL Query Specification

This appendix documents the constraint-to-SQL translation rules and provides representative query examples. Alignment with the main-text formalization: π_{topo} and π_{precond} compile to WHERE filters; for relational checks, π_{schema} is evaluated per *comparison group* (the guarded tuples sharing the grouping key, e.g., all replicas of one parameter at one step) and appears as the HAVING predicate—matching Eq. (violation set) and the Online Detection section of the main paper. Single-tuple checks are the singleton special case of the same shape.

I.1 Database Schema

Database Schema

```
1 CREATE TABLE coredump (
2   step    INTEGER,    -- Training step index
3   stage   TEXT,       -- Training stage identifier
4   data    JSON        -- Tensor metadata as JSON
5 );
```

I.2 JSON Field Structure

I.3 Representative SQL Queries

I.3.1 DP Parameter Consistency Check.

DP Parameter Consistency Check

```
1 WITH param_stats AS (
2   SELECT
3     json_extract_string(data, '$.name') as param_name,
4     CAST(json_extract_string(data, '$.dp') AS INTEGER) as dp_rank,
5     json_extract_string(data, '$.cksum') as cksum,
6     step
7   FROM coredump
```

```

8     WHERE stage = 'model-after-optimizer-step'
9     AND json_extract_string(data, '$.cksum') IS NOT NULL
10 ),
11 inconsistency_check AS (
12     SELECT
13         param_name, step,
14         COUNT(DISTINCT cksum) as distinct_cksums,
15         array_agg(DISTINCT dp_rank) as dp_ranks
16     FROM param_stats
17     GROUP BY param_name, step
18     HAVING COUNT(DISTINCT cksum) > 1
19 )
20 SELECT * FROM inconsistency_check ORDER BY step, param_name;

```

I.3.2 TP Weight Partition Check.

TP Weight Partition Check

```

1 WITH tp_params AS (
2     SELECT
3         json_extract_string(data, '$.name') as param_name,
4         CAST(json_extract_string(data, '$.tp') AS INTEGER) as tp_rank,
5         json_extract_string(data, '$.cksum') as cksum
6     FROM coredump
7     WHERE stage = 'model-after-backward'
8     AND json_extract_string(data, '$.name') LIKE '%kv%'
9 )
10 SELECT param_name, COUNT(DISTINCT cksum) as unique_shards
11 FROM tp_params
12 GROUP BY param_name
13 HAVING COUNT(DISTINCT cksum) = 1; -- Should be different across TP ranks

```

J Data Collector: Implementation Details

Implementation detail. Supports the Data Collector / Efficiency claims in the main text; skip if only checking detection or mining soundness. This appendix documents the VTimeline-based data collection infrastructure, including hook injection points and the complete dump interface.

J.1 Hook Injection Points

J.2 MegatronCollector API

MegatronCollector API

```

1 class MegatronCollector:
2     @classmethod
3     def set_process_group_info(cls, ranks_info: Dict[str, int]):
4         """Set parallel rank information (dp, tp, pp, cp, ep)."""
5
6     @classmethod
7     def set_core(cls, model, optimizer, scheduler):
8         """Register model, optimizer, and scheduler for tracking."""
9
10    @classmethod
11    def dump_model(cls, stage_name: str):
12        """Dump all model parameters with checksums and metadata."""
13
14    @classmethod
15    def dump_main_grad(cls, param, param_name: str, stage_name: str):
16        """Dump main_grad for a specific parameter."""
17
18    @classmethod
19    def dump_optimizer_state(cls, stage_name: str):

```

Algorithm 2 SECheck One-Click Detection**Require:** Database path db , Megatron script path $script$ **Ensure:** Detection report R

```

1: // Phase 1: Configuration Parsing
2:  $config \leftarrow \text{PARSESCRIPT}(script)$ 
3:  $schema \leftarrow \text{ANALYZESchema}(db)$ 
4: // Phase 2: Constraint Filtering
5:  $C \leftarrow \text{LOADCONSTRAINTS}()$ 
6:  $C_{app} \leftarrow \text{FILTER}(C, config)$ 
7: // Phase 3: Batch Verification
8:  $R \leftarrow \emptyset$ 
9: for all  $c \in C_{app}$  do
10:    $sql \leftarrow \text{GENSQL}(c, schema)$ 
11:    $result \leftarrow \text{EXECUTE}(db, sql)$ 
12:    $R \leftarrow R \cup \{\text{ANALYZE}(c, result)\}$ 
13: end for
14: return  $\text{GENERATEREPORT}(R)$ 

```

```

20:         """Dump optimizer internal states (exp_avg, exp_avg_sq, etc.)."""
21:
22:     @classmethod
23:     def step(cls):
24:         """Increment the global step counter."""

```

J.3 Extended Statistics Collection

In addition to checksums, the collector captures distribution statistics for advanced consistency checks:

- **Quantiles:** 25th, 50th, 75th percentiles
- **Higher-order moments:** Skewness, Kurtosis
- **Distribution shape:** Histogram checksum, Entropy

K Batch Detection: One-Click Execution

Implementation detail. Reproducibility scripts for the online verifier; not required to validate Real-SE detection rates reported in the main text.

This appendix documents the automated batch detection workflow implemented in SECheck.

K.1 SECheck Main Workflow

Algorithm 2 presents the one-click detection workflow. Given a database path db containing traced tensors and a Megatron launch script $script$, the algorithm automatically filters applicable constraints and executes batch verification.

K.2 Batch Execution Script**K.3 Execution Flow****Execution Flow**

```

1: # Batch execution command
2: ./batch_secheck_d.sh [optional: db_directory]
3:
4: # For each DB directory:
5: # 1. Locate corresponding injection script
6: # 2. Find Collector subdirectory
7: # 3. Execute: secheck-merge --auto-evaluate <script> --hierarchical
8: # 4. Log results to secheck_logs/

```

L Diagnosis Accuracy

Detecting an error alone offers limited value to engineers—they also need to know *which constraint was violated* and *at which layer the fault manifests*. This appendix answers two questions: (i) when one of TRAINAUDIT’s checks fires, does the check itself name the fault’s semantic class? (ii) for complex faults that propagate across multiple layers, can the drill-down logic converge to the root cause?

Setup. TRAINAUDIT’s diagnosis chain consists of two stages: (1) *rule names as fault signatures*—each rule’s name (e.g., `optim-step-counter-monotonic`) directly encodes “which object should satisfy which condition”; (2) *C1 expander + C2 drill-down logic*—the C1 expander walks the constraint hierarchy backward along the “upstream dependency” template to expand a candidate source-tensor set, and the C2 drill-down logic uses runtime evidence (first-NaN step, stride offset relative to `zero_grad`, etc.) to converge to a root cause within the expanded set. We measure this chain on the 17 Real-SE cases detected in the main paper’s detection study (the boundary case triggers no rule and is excluded). For each case we record the candidate-set size $|S_{\text{rule}}|$ at rule firing time, $|S_{C1}|$ after the C1 expander, and $|S_{C2}|$ after the C2 drill-down, taking the fix-patch-annotated true root cause ($|\text{set}| = 1$) as ground truth.

Finding 1: Rule names alone converge 53% of the cases at the alert stage. Figure L.1 shows that **9 of the 17 detected cases (53%)** have $|S_{\text{rule}}| = 1$ the moment the rule fires—the rule name alone localizes the fault to a unique object. For example, `optim-step-counter-monotonic` immediately tells engineers that “`optimizer.state['step']` is no longer monotonically increasing”; `olmo_emb_init_std_sqrt_dmodel_invariant` directly pinpoints “the embedding initialization std lacks the $\sqrt{d_{\text{model}}}$ scaling”; `checkpoint-preserve-rng` points to “checkpoint resume not preserving RNG state.” Every rule is required to carry an expressive name at generation time—naming is equivalent to writing the protected constraint in natural language. By contrast, TrainCheck’s relation templates can only report “two variables disagree,” forcing engineers to manually infer the fault’s semantic identity from reference traces.

Finding 2: C1+C2 compresses the search space by $\sim 6\times$ on the 47% ambiguous cases. The remaining 8 cases (47%) are ambiguous at the rule-alert stage—the alert involves multiple candidate ranks/tensors, or lands on a downstream object while the true root cause lies upstream. The C1 expander expands the candidate set along the “upstream dependency” template to a median of **6** (peak 12); the C2 drill-down then uses runtime evidence to converge. 7 of these cases (5 by C1, 2 further by C2) reach $|\text{set}| = 1$; only 1 case (6%) remains ambiguous after the full pipeline, requiring signals beyond the online trace to disambiguate. The “expand-contract” shape is the signature of this mechanism: the rule name points to the symptom, C1 lays out all potential source objects, and C2 prunes using dynamic evidence.

Finding 3 (case study): Multi-hypothesis diagnosis for the BF16+ZeRO-0 dual-bug. H5—a BF16+ZeRO-0 dual-bug on DeepSpeed v0.18.7, reproduced end-to-end outside the Real-SE set—converges along a three-step chain: (1) the A0 rule `no-nan-inf` fires successively at steps 8–14 on the global gradient after `Optimizer.step`; (2) the C1 expander lays out 6 candidate source tensors grouped by DP rank, five of which already had NaNs, the earliest on rank 0’s layer `0.bias`; (3) the C2 drill-down observes a stride offset between NaN steps and `bf16_optimizer.zero_grad` calls, triggering the probe rule `multi-backward-per-step-fragile-config`—the dual-bug’s root cause (`zero_grad` missed under multi-backward configuration). The diagnosis report completes within 4 seconds. Both Naïve Monitoring and TrainCheck can report “NaN appeared,” but neither penetrates to the semantic assumption of `zero_grad`.

Finding 4: TrainCheck’s schema rigidity makes diagnosis fail on heterogeneous traces. TrainCheck’s invariants are bound to specific schema columns of the reference trace; when the deployment trace lacks these columns, or the fault signature falls outside the trace templates, the rules cannot fire and diagnosis accuracy drops to 0% on those instances. TRAINAUDIT circumvents this through *schema adaptability*—the verifier dynamically reads the DuckDB schema at every check and synthesizes SQL against the available attributes (falling back to shape or checksum when statistical aggregates are missing). TrainCheck regards trace schema as part of its invariants; TRAINAUDIT regards it as runtime context.

M Production Case Studies

We trace three silent-error cases end-to-end through TRAINAUDIT’s pipeline—from template instantiation through instrumentation to a non-empty query at detection time. The three cases respectively cover the schema, topology/metadata, and behavior/optimizer-boundary signal classes. Figure M.1 summarizes the diagnostic signals.

Case 1: SFT tokenizer prompt-format misconfiguration. Case 1 is a newly observed in-production silent error outside the 392-bug survey, encountered when retraining a 7B coding model under a refactored SFT pipeline. The launch script defaults `--sft-tokenizer-prompt-format` to `nemotron-h-aligned`, whose chat-template emits role delimiters such as `<SPECIAL_10>`. The tokenizer’s vocabulary contains no such tokens, so the BPE backend silently shatters each delimiter into six subword pieces. No exception is raised; the training loss decreases monotonically and is bitwise indistinguishable from a correctly configured reference run (Fig. M.1, top-left), but the model is learning a corrupted dialog protocol. The damage is visible only downstream: SWE-bench pass rate plateaus at 28.8% versus 44.5% for the corrected configuration. TRAINAUDIT mines the rule `SFT_CHAT_TEMPLATE_VOCAB_INTEGRITY` from the tokenizer-rendering call site: the schema predicate asserts that every role-delimiter string emitted by the chat-template renderer maps to a single vocabulary id; the topology predicate is trivially true (the fault is topology-independent); the precondition restricts the check to the SFT phase, after tokenizer initialization and before the first forward pass. The verifier flags the violation at step 1. This case validates that the three-predicate framework is not over-fit to the 392-bug pool: any fault admitting a runtime-observable three-predicate factoring—including the degenerate topology case—is detectable, even when the bug sits outside distributed-runtime semantics.

Case 2: Megatron-LM Muon+PP+MTP optimizer routing. On a Megatron-LM HEAD configuration using Muon, pipeline parallelism, and MTP, the two PP replicas of the tied `word_embeddings` must share the same optimizer routing. In the actual run, the replica on the MTP-only last PP stage is missing the `is_embedding_or_output_parameter=True` flag and gets added to the Muon parameter group, while the PP

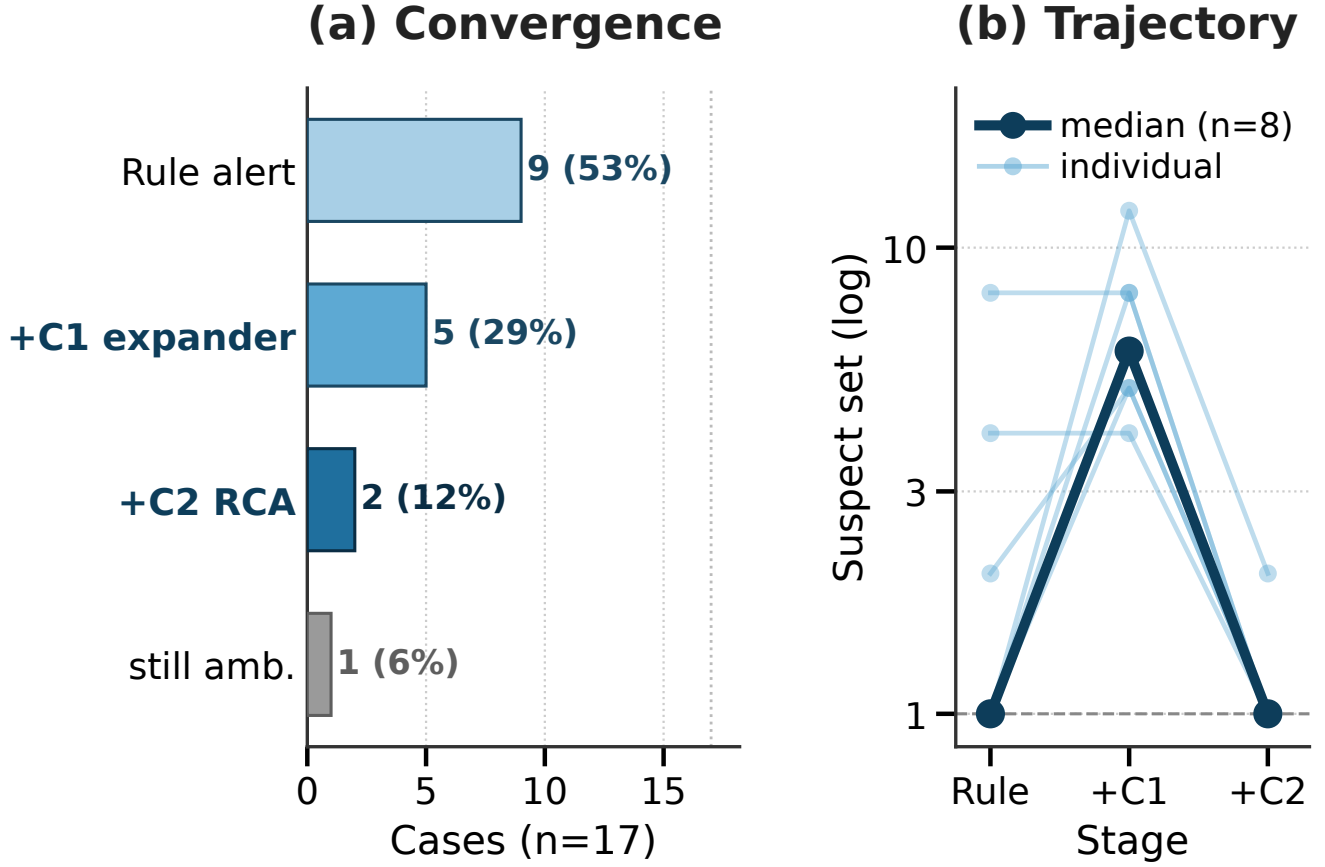


Figure L.1: C1+C2 drill-down mechanism on 17 detected Real-SE cases: 9/17 converge at rule alert, 5/17 require C1, 2/17 require C2; the median candidate-set trajectory on the 8 ambiguous cases is $1 \rightarrow 6 \rightarrow 1$.

rank 0 replica is still updated by AdamW—two semantically equivalent replicas receiving different update rules. The fault raises no exception and the loss curve remains smooth (Fig. M.1, middle), but the verifier observes in the init-time snapshot that the same `tied_group_id=word_embeddings` maps to an optimizer set of {AdamW, Muon} (Figure M.2). TRAINAUDIT fires MEGATRON_TIED_EMBED_OPT_ROUTING before the first optimizer step; maintainers confirmed and fixed the bug upstream.

Case 3: DeepSpeed ZeRO-offload multi-backward gradient loss. Under DeepSpeed v0.18.7 ZeRO-2 + CPU offload + gradient-accumulation steps=1, multiple backward passes execute within a single training step before one optimizer step. The gradients from all 4 backward passes should accumulate, but the buggy path retains only the last backward—the previous 3 are silently discarded. Policy loss still descends smoothly (Fig. M.1, bottom); the end-to-end oracle shows the bug-path total parameter change (2.479887) exactly matches the last-only oracle, while the all-4-grad oracle is 7.146901 (2.88×). TRAINAUDIT encodes this as the behavior/optimizer-boundary rule `multi-backward-per-step-fragile-config`, firing before the optimizer step. Some silent errors manifest not as a tensor value out of range, but as a violation of cross-hookpoint call-boundary semantics.

N Motivation Figures and Benchmark Coverage

Table N.1 maps each descriptive subsystem label to a representative Real-SE case.

O Extended Experimental Data

O.1 Per-Case Detection Results

O.2 Schema Tier Coverage–Overhead Curve

O.3 Rule Library and Hookpoint Deployment Overview

Appendix D documents the frozen experimental snapshot and its held-out validation. The evaluated TRAINAUDIT deployment snapshot on Megatron-LM contains 24 active rules (12 A0 PyTorch public-interface rules + 9 A1 framework-adapter rules + 3 active probes), bound to 8 hookpoints: `optim.step.{pre,post}`, `module.forward.{pre,post}`, `checkpoint.{save,load}`, `distributed.all_reduce`, and `build.snapshot`. These counts are deployment-version measurements, not architectural constants. Catalog relations without a concrete framework hook remain offline candidates until the Invariant Miner grounds and verifies them. The rule-to-hookpoint index is automatically maintained at every tick by the hookpoint-aware verifier, compressing the per-tick evaluation cost to an average of 1–3 rules.

References

- [1] Junjie Chen, Yihua Liang, Qingchao Shen, Jiajun Jiang, and Shuochuan Li. 2023. Toward Understanding Deep Learning Framework Bugs. *ACM Trans. Softw. Eng. Methodol.* 32, 6 (2023), 156:1–156:32.
- [2] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering Denial Constraints. *Proceedings of the VLDB Endowment (PVLDB)* 6, 13 (2013), 1498–1509. doi:10.14778/2536258.2536262
- [3] Michael D Ernst, Jeff H Perkins, Philip J Guo, Stephen McCamant, Carlos Pacheco, Matthew S Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *Science of computer programming* 69, 1-3 (2007), 35–45. doi:10.1016/j.scico.2007.01.014
- [4] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional Functional Dependencies for Capturing Data Inconsistencies. *ACM Transactions on Database Systems (TODS)* 33, 2 (2008), 6:1–6:48. doi:10.1145/1366102.1366103
- [5] Stewart Grant, Hendrik Cech, and Ivan Beschastnikh. 2018. Inferring and asserting distributed system invariants. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg, Sweden, 1149–1159. doi:10.1145/3180155.3180174
- [6] Hao Guan, Ying Xiao, Jiayang Li, Yepang Liu, and Guangdong Bai. 2023. A Comprehensive Study of Real-World Bugs in Machine Learning Model Optimization. In *Proceedings of the 45th International Conference on Software Engineering (Melbourne, Victoria, Australia) (ICSE '23)*. IEEE Press, 147–158. doi:10.1109/ICSE48619.2023.00024
- [7] Haryadi S Gunawi, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tirat Patana-anake, Thanh Do, Jeffrey Adber, Shan Lu, et al. 2014. What Bugs Live in the Cloud? A Study of 3000+ Issues in Cloud Systems. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. ACM, Seattle, WA, USA, 1–14. doi:10.1145/2670979.2670986
- [8] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoukjoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019), 103–112. <https://papers.nips.cc/paper/8717-gpipe-efficient-training-of-giant-neural-networks-using-pipeline-parallelism>
- [9] Haitian Jiang, Shaowei Zhu, Zhen Zhang, Zhenyu Song, Xinwei Fu, Zhen Jia, Yida Wang, and Jinyang Li. 2025. TTrace: Lightweight Error Checking and Diagnosis for Distributed Training. *arXiv preprint arXiv:2506.09280* abs/2506.09280 (2025), 14 pages. arXiv:2506.09280 [cs.DC] <https://arxiv.org/abs/2506.09280>
- [10] Yuxuan Jiang, Ziming Zhou, Boyu Xu, Beijie Liu, Runhui Xu, and Peng Huang. 2025. *Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks*. Technical Report. University of Michigan. A shorter version of this paper is to appear in the Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25), July, 2025. arXiv:2506.14813 <https://arxiv.org/abs/2506.14813>
- [11] Ziheng Jiang, Haibin Lin, Yimin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*.
- [12] Tanakorn Leesatapornwongsa, Jeffrey F Lukman, Shan Lu, and Haryadi S Gunawi. 2016. TaxDC: A Taxonomy of Non-Deterministic Concurrency Bugs in Datacenter Distributed Systems. In *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, Atlanta, GA, USA, 517–530. doi:10.1145/2872362.2872372
- [13] Dmitry Lepikhin, Hyoukjoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations*.
- [14] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. PyTorch distributed: experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3005–3018. doi:10.14778/3415478.3415530
- [15] Gaoxiang Liu, Tao Chen, Yanlin Wang, and Yepang Liu. 2024. Investigating and Detecting Silent Bugs in PyTorch Programs. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE.
- [16] Yunchi Lu, Youshan Miao, Cheng Tan, Peng Huang, Yi Zhu, Xian Zhang, and Fan Yang. 2025. TrainVerify: Equivalence-Based Verification for Distributed LLM Training. arXiv:2506.15961 [cs.LG]
- [17] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S Vetter. 2018. Nvidia tensor core programmability, performance & precision. In *2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW)*. IEEE, IEEE, Vancouver, BC, Canada, 522–531. doi:10.1109/IPDPSW.2018.00091
- [18] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2018. Mixed Precision Training. In *International Conference on Learning Representations*. OpenReview.net, Vancouver, BC, Canada, 10 pages. <https://openreview.net/forum?id=r1gs9JGRZ>
- [19] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*. Association for Computing Machinery, St. Louis, Missouri, 1–15.
- [20] Rahul Pandita, Xusheng Xiao, Hao Zhong, Tao Xie, Stephen Oney, and Amit Paradkar. 2012. Inferring Method Specifications from Natural Language API Descriptions. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. IEEE, Zurich, Switzerland, 815–825. doi:10.1109/ICSE.2012.6227119
- [21] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, Honolulu, HI, USA, 27496–27520. <https://proceedings.mlr.press/v202/pei23a.html>
- [22] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, IEEE, Atlanta, GA, USA, 1–16. doi:10.1109/SC41405.2020.00040
- [23] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. ACM, Virtual Event, 3505–3506. doi:10.1145/3394486.3406703
- [24] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. 2018. Automating Large-Scale Data Quality Verification. *Proceedings of the VLDB Endowment (PVLDB)* 11, 12 (2018), 1781–1794. doi:10.14778/3229863.3229867
- [25] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. In *Proceedings of the International Conference on Learning Representations (ICLR)*. OpenReview.net, New Orleans, LA, USA, 13 pages. ICLR 2019. <https://arxiv.org/abs/1909.08053>

- [26] Jie Song and Yeye He. 2021. Auto-Validate: Unsupervised Data Validation Using Data-Domain Patterns Inferred from Data Lakes. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, Xi'an, China, 1678–1691. doi:10.1145/3448016.3457250
- [27] Florian Tambon, Amin Nikanjam, Le An, Foutse Khomh, and Giuliano Antoniol. 2023. Silent bugs in deep learning frameworks: an empirical study of Keras and TensorFlow. *Empirical Softw. Engg.* 29, 1 (Nov. 2023), 34 pages. doi:10.1007/s10664-023-10389-6
- [28] Naigang Wang, Jungwook Choi, Daniel Brand, Chia-Yu Chen, and Kailash Gopalakrishnan. 2018. Training deep neural networks with 8-bit floating point numbers. *Advances in neural information processing systems* 31 (2018), 7675–7684. <https://papers.nips.cc/paper/7997-training-deep-neural-networks-with-8-bit-floating-point-numbers>
- [29] Yuhao Zhang, Yifan Chen, Shing-Chi Cheung, Yingfei Xiong, and Lu Zhang. 2018. An Empirical Study on TensorFlow Program Bugs. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Amsterdam, Netherlands, 129–140. doi:10.1145/3213846.3213865
- [30] Yiteng Zhang, Zirui Liu, Zhiqiang Zang, Keyan Guo, Haoxiang Li, Wencong Xiao, and Mengwei Xu. 2024. Verifying Semantic Equivalence of Large Models with Equality Saturation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*. ACM, 267–279.

Table A.1: Extended comparison with related work. The key distinction is the source of correctness information: outcome metrics, execution traces, reference executions, static plans, or bug-derived semantic patterns.

Area	Representative work	Main oracle
Empirical bug studies	TaxDC [12], cloud-system bug studies [7], TensorFlow bug studies [1, 29], ML optimization bugs [6]	Bug taxonom

Table B.1: Separation of the three experimental datasets. The catalog study and the detector benchmark apply different filters to the same evidence corpus.

Stage	Input	Construction rule	Output
Evidence corpus	295-record pool + 96 records unique to the 128-record pool + 1 orphan	Retain each of the 32 shared incidents once; exclude an incomplete external orphan before assembly	392 records
Catalog induction	392 records	Incident-grouped, framework/class-stratified seed/dev/test split; Real-SE forced into locked test	128 / 186 / 78
Detector benchmark	392 records	Class/framework coverage targets plus commit pair, runnable driver, archived buggy/fixed evidence, and no surrogate	17 + 1 boundary

Table B.2: Distribution of the 4-tier silent-evidence rating on the 392-record corpus. L1 provides physical buggy/fixed evidence; L2–L3 are textual evidence levels; L4 records have sparse evidence and are retained only as a review queue. Source: `silent_evidence_392.json`.

Tier	Criterion	Count	Note
L1	reproduced + expected_output shows silent	23	physical ground truth
L2	root_cause has silent keywords, no crash keywords	161	—
L3	only issue title carries silent signal	72	—
L4	sparse fields	136	weakest tier; manual review required
<i>Loud-bug counter-evidence</i>			
	available metadata signals a newly added raise/assert	0 recorded	supporting evidence only
	crash-keyword WARNING (retained after manual review)	2/392 (0.5%)	both false positives

Table B.3: Inter-rater reliability across the two protocols. Protocol A measures re-annotation agreement on 32 doubly-and-independently annotated bugs; Protocol B uses an independent LLM rerater on a 50-bug stratified sample to compute Cohen’s κ . Source: `irr_50_results.json`.

Protocol	Sample	Metric	Value	Interpretation
A (Re-annotation)	32 bugs	raw agreement	78.1% (25/32)	high
B (Independent LLM)	50 stratified	Cohen’s κ	0.566	moderate (L&K ’77)
B	50 stratified	raw agreement	60.0% (30/50)	—
B	50 stratified	p_{expected}	0.078	label-vocabulary baseline
<i>Literature benchmark</i>				
DeepLearningBench (ICSE’20)	—	κ	0.59	8 classes
PROMISE series	—	κ	0.55–0.75	6–10 classes

Table D.1: Frozen snapshot used in the template-induction study. Schema coverage counts cases matched to a template in that snapshot; joint coverage additionally requires topology and validity guards to be instantiable. Counts describe this experiment, not a fixed system-wide catalog size.

Stage	Cases	Operators	Templates	Result
Seed construction	128	18	27	Initial catalog snapshot
Development completion	186	20	35	No core relation revised; saturation rule not reached
Locked test	78	—	—	72/78 schema match; 71/78 joint grounding; 3 new singletons

Table F.1: Timeline of B10 Error Detection

Step	Component	Action
1	Data Collector	Captures <code>micro_step_id</code> state at each micro-batch
2	SQL Agent	Generates query to detect counter inconsistency
3	Verifier	Executes SQL, identifies anomalous records
4	Analysis Agent	Correlates counter state with gradient operation type
5	Report	Localizes root cause to initialization logic

Table G.1: Operational states (S_1 - S_5) and agent responsibilities in the Invariant Miner. *Logical names match the main-text description of the five logical agents; the underlying prompt-implementation names (Coordinator / Aggregation Expert / Training Expert / Write Agent / Report Agent) appear in Table H.1.*

State	Agent	Role
S_1	Gap-Analysis Agent	Gap analysis, target selection
S_2	Evidence Agent	Evidence retrieval from documentation
S_3	Synthesis Agent	Constraint synthesis with bidirectional verification
S_4	Persistence Agent	JSON formatting, persistent storage
S_5	Reporting Agent	Iteration summary, repository statistics

Table G.3: FSM State Transition Rules

Current State	Trigger Keyword	Next State
S_1 (Gap Analysis)	CONTINUE_AGGREGATION	S_2
S_2 (Ext. Research)	research_submitted=True	S_3
S_3 (Synthesis)	TIGHTEN_SCOPE	S_3 (self-loop)
S_3 (Synthesis)	FETCH_CONTRA_EVIDENCE	S_2 (loop)
S_3 (Synthesis)	WRITE_CONSTRAINT	S_4
S_3 (Synthesis)	REJECT	S_5
S_4 (Write)	write_complete=True	S_5
S_5 (Reporting)	feedback_logged=True	S_1

Table H.1: Summary of Agent Prompts

Agent	Tokens	Temp.	Key Features
Training Expert	~3000	0.1	4-stage funnel, equivalence detection
Aggregation Expert	~1500	0.2	5-step verification, source filtering
Coordinate Agent	~2000	0.1	Routing rules, lineage validation
Write Agent	~1000	0.1	JSON formatting, persistence
Report Agent	~500	0.2	Statistics aggregation

Table I.1: JSON Data Field Specification

Field	Type	Description
name	string	Parameter name (e.g., <code>decoder.layers.0.attention.qkv.weight</code>)
cksum	string	SHA-256 checksum of tensor data
shape	array	Tensor dimensions (e.g., <code>[4096, 4096]</code>)
type	string	Tensor dtype (e.g., <code>torch.cuda.BFloat16Tensor</code>)
requires_grad	boolean	Whether parameter requires gradient
dp/tp/pp/cp/ep	integer	Parallel rank indices
grad_cksum	string	Checksum of gradient tensor (nullable)

Table J.1: Complete List of Hook Injection Points

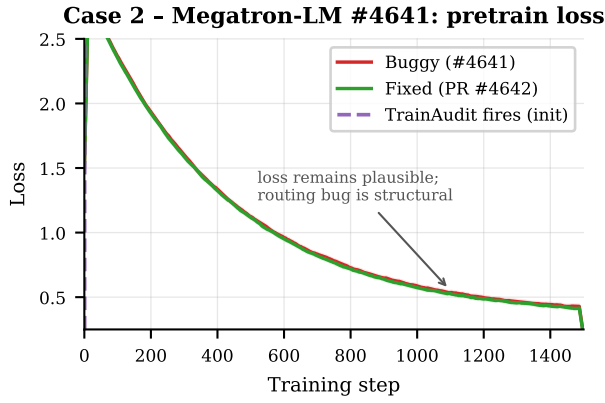
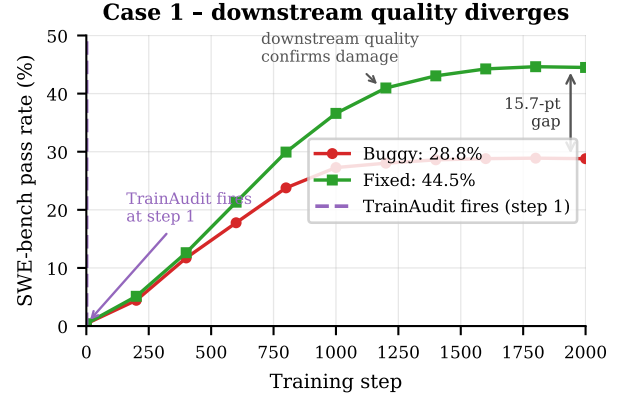
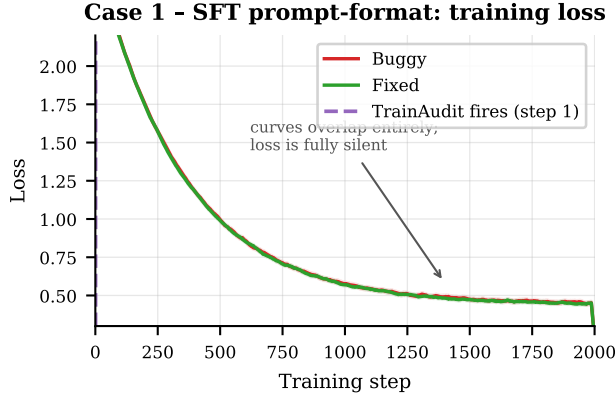
Stage Name	Injection Location	Data Captured
model-before-forward	Before <code>model.forward()</code>	Parameters, shapes
model-after-forward-mbs-*	After each micro-batch forward	Activations (optional)
main-grad-in-backward	During gradient computation	Main gradients
model-after-backward	After <code>loss.backward()</code>	Gradients, <code>requires_grad</code>
model-before-optimizer-step	Before <code>optimizer.step()</code>	Optimizer states
model-after-optimizer-step	After <code>optimizer.step()</code>	Updated parameters
after-get-batch	After data loading	Input tensors

Table K.1: Representative DB-Script Mappings (40+ total)

Database Directory	Injection Script
	<i>Data Parallel (DP) Checks</i>
optimizer_state_test_db	<code>pretrain_inject_optimizer_state.sh</code>
main_grad_test_db	<code>pretrain_inject_main_grad.sh</code>
requires_grad_test_db	<code>pretrain_inject_requires_grad.sh</code>
dtype_test_db	<code>pretrain_inject_dtype.sh</code>
	<i>Tensor Parallel (TP) Checks</i>
tp_grad_same_test_db	<code>pretrain_inject_tp_grad_same.sh</code>
tp_attn_proj_test_db	<code>pretrain_inject_tp_attn_proj.sh</code>
tp_layernorm_test_db	<code>pretrain_inject_tp_layernorm.sh</code>
tp_embedding_test_db	<code>pretrain_inject_tp_embedding.sh</code>

Table N.1: Real-SE diversity across the descriptive subsystem labels. Each label has at least one representative case; this is not a claim of exhaustive fault-space coverage.

Primary label	Representative Real-SE case
Numerical	OLMo-ce5f30d5 (emb-init-std-sqrt-dmodel)
Control Flow	OLMoC-2b6cf996 (optim-step-counter-monotonic)
Grad Sync	Megatron-3c637fc0d (replica-cksum-equal)
Checkpoint	OLMo-204ad53c (checkpoint-preserve-rng)
MoE	Megatron-e6d56d68 (invocation-frequency-per-step)
Optim State	Megatron-1505db4c (chained-optimizer-param-groups-resync)
Data Loading	OLMo-167a7ac8 (data-loader-token-id-range)
Communication	DS-8cbbb53 (process-group-size-correct)
Dtype	DS-d56268f3 (comm-dtype-matches-training)
Loss Comp.	OLMo-a57f3803 (zloss-mean-reduction)
Sharding	Megatron-99f999a4 (layer-count-strict)
LR Schedule	OLMoC-6e330ba2 (initial-lr-present)
Offload	DS-e6e8c137 (moe-cpu-offload grad-norm device)



Case 2 - verifier output @ init

snapshot: tied_group_id = word_embeddings

PP rank	embed tag	optimizer
rank 0	yes	AdamW
rank N (MTP-only)	no	Muon

Expected: one tied group -> one optimizer rule
Observed: {AdamW, Muon}

FIRE before first optimizer step

Megatron-LM #4641 / PR #4642

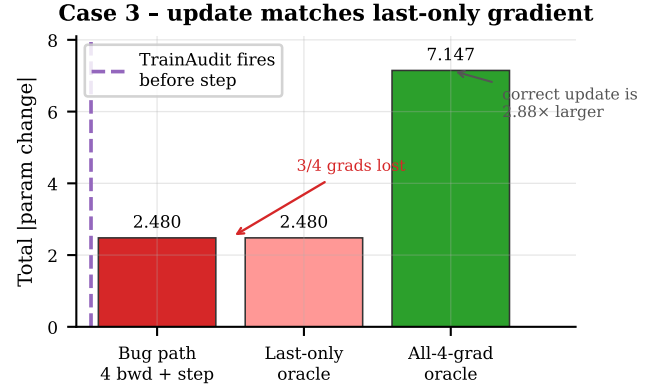
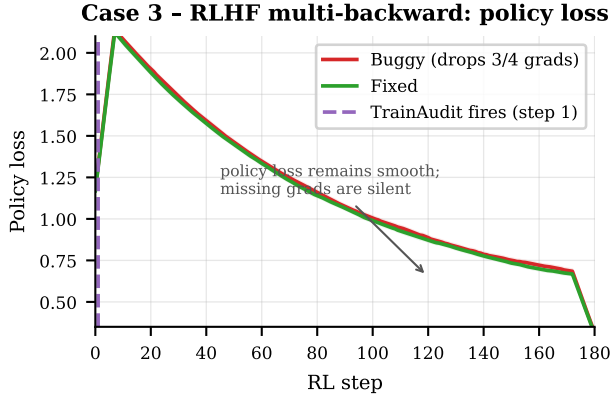


Figure M.1: Diagnostic signals for three production silent-error cases. *Left column*: training loss; *right column*: a framework-specific auxiliary signal. The purple dashed line marks the step at which TRAINAUDIT fires. Case 1 (SFT prompt-format misconfiguration, top): the training loss of the buggy and fixed runs overlap end-to-end across 2,000 steps—the loss signal is fully silent. Yet the downstream SWE-bench pass rate diverges from ~step 400 onward, with the buggy run plateauing at 28.8% while the fixed run climbs to 44.5% (a 15.7-point gap). TRAINAUDIT fires a schema-predicate rule at step 1. Case 2 (Megatron-LM Muon+PP+MTP, middle): the loss remains smooth; but the verifier’s init-time snapshot shows the two PP replicas of the tied `word_embeddings` routed to different optimizer groups—PP rank 0 to AdamW, the MTP-only last rank to Muon. TRAINAUDIT fires the optimizer-routing constraint before the first optimizer step. Case 3 (DeepSpeed ZeRO-offload multi-backward, bottom): policy loss still descends smoothly; but the parameter-update magnitude matches the last-only oracle exactly (2.479887), while the correct all-4-grad oracle is 7.146901 (2.88×), indicating that the first 3/4 backward gradients are silently dropped.

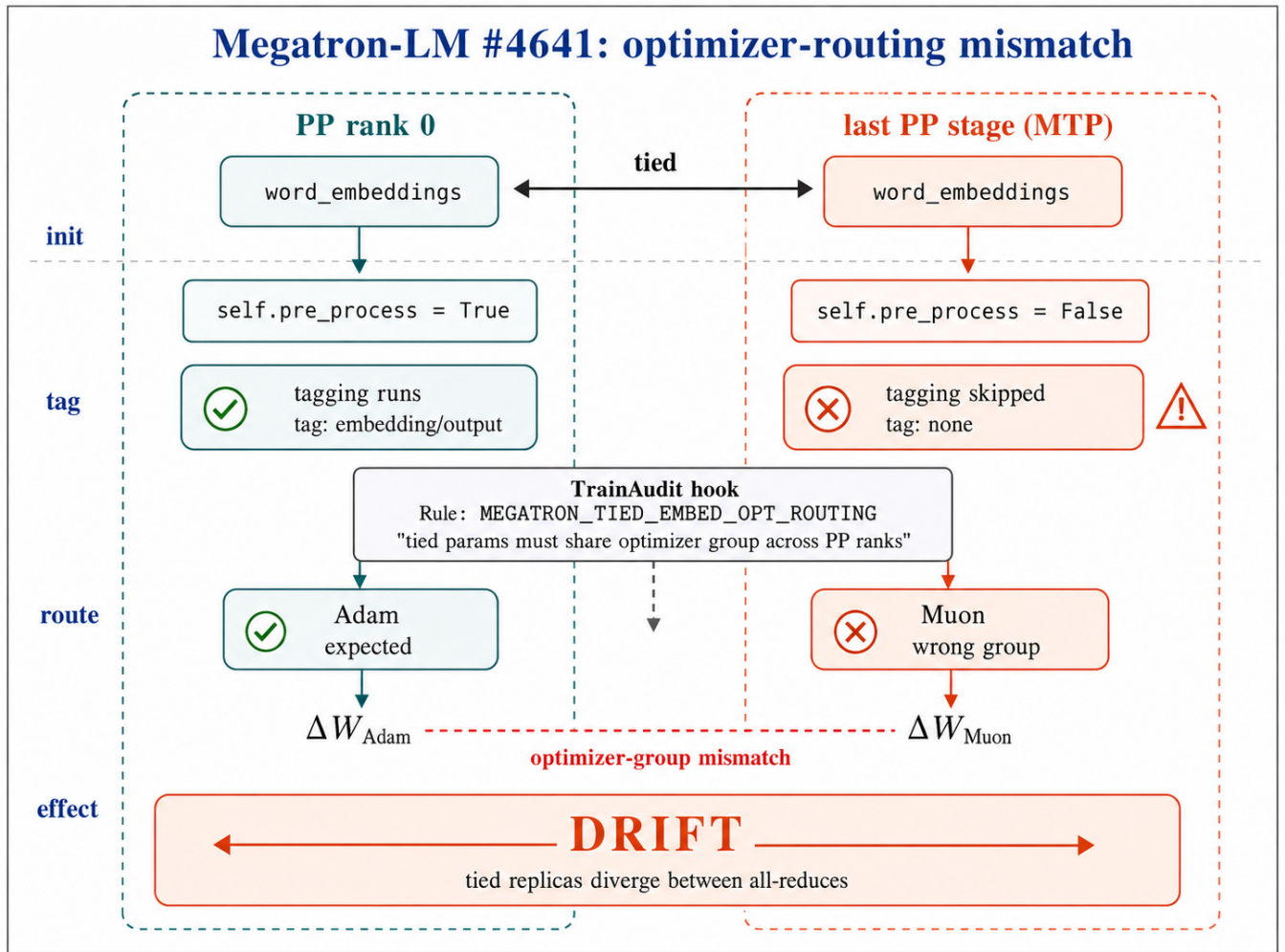


Figure M.2: Megatron-LM (Muon+PP+MTP): tied `word_embeddings` on the MTP-only last PP stage is silently routed to Muon while the PP rank 0 replica stays on Adam. TRAINAUDIT catches this at param-group registration via MEGATRON_TIED_EMBED_OPT_ROUTING; confirmed and fixed upstream.

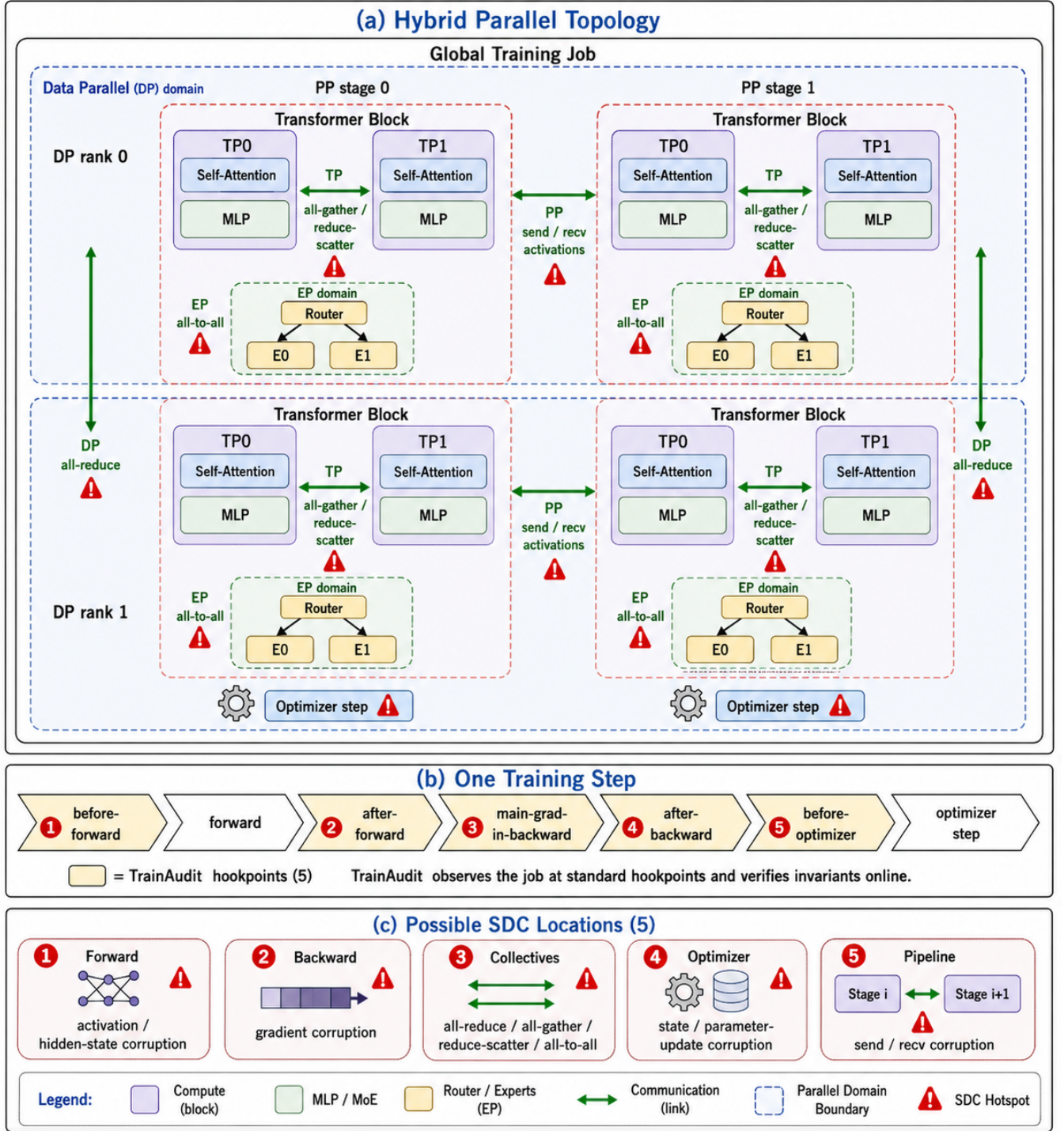


Figure N.1: Panorama of hybrid-parallel LLM training and where silent errors strike. (a) A single global training job is sharded across four orthogonal parallelism dimensions—data (DP), pipeline (PP), tensor (TP), and expert (EP)—each with its own collective. (b) A single training step exposes five standard hookpoints at which TRAINAUDIT observes runtime state. (c) Five silent-error hotspots aligned with those hookpoints. Red triangles in (a) mark the same hotspots in the topology view.

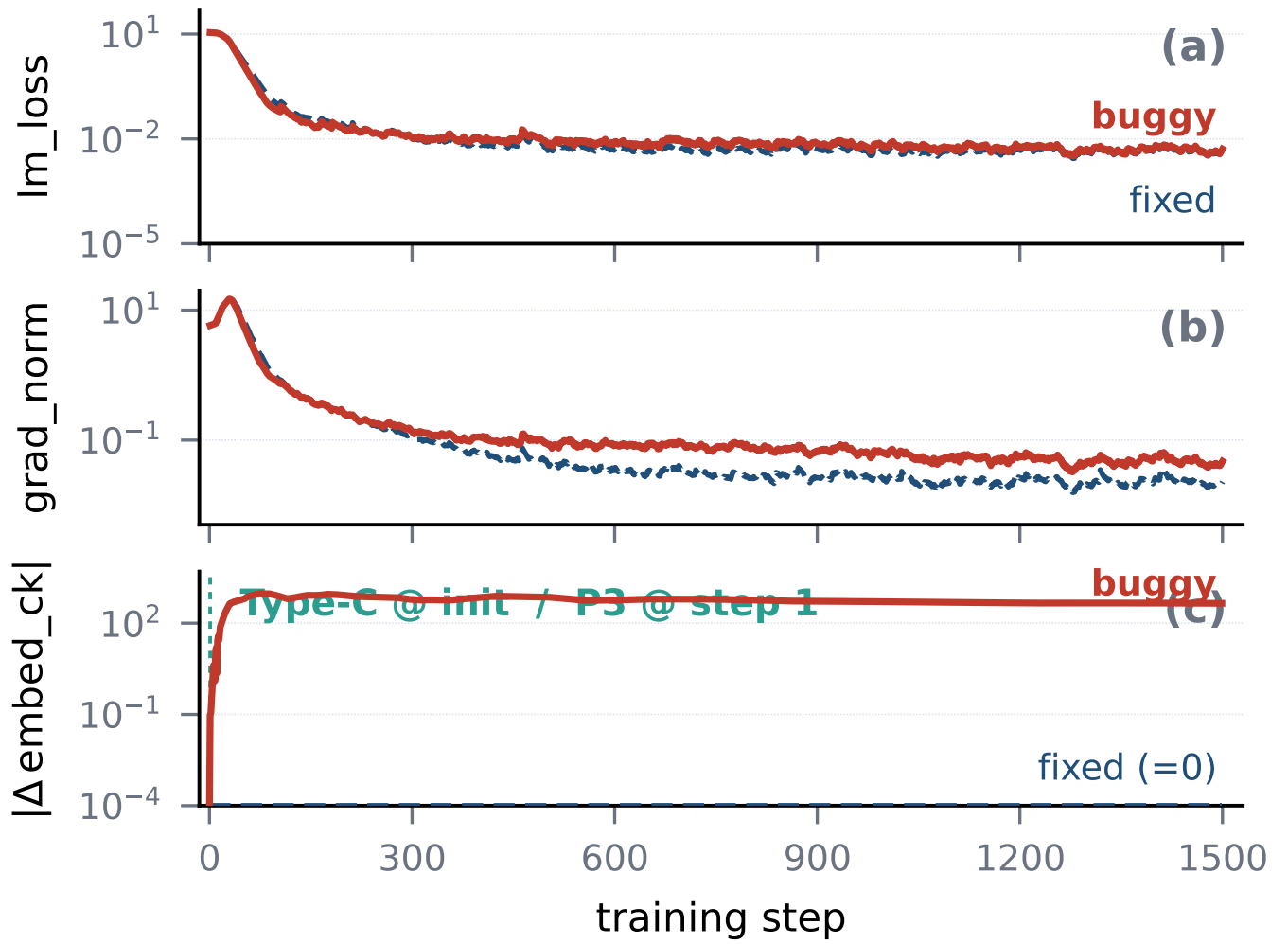


Figure N.2: Silent error impact on training metrics. Loss (a) and gradient norm (b) both appear normal for hundreds of steps after corruption is introduced. The gap is *detection latency*—during which GPU-hours are wasted on a corrupted trajectory.

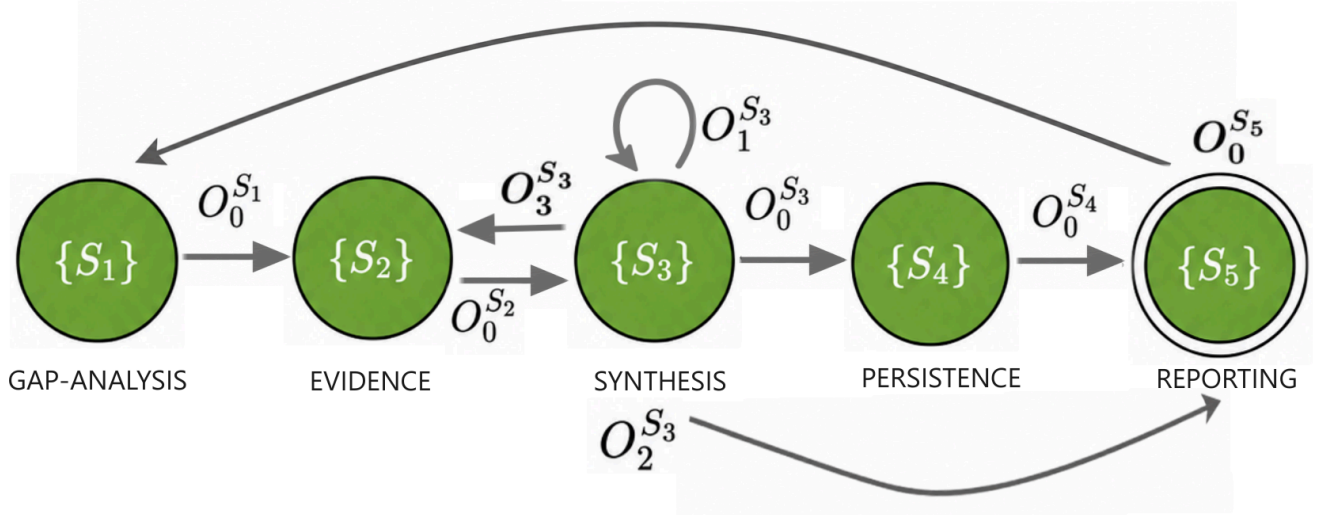


Figure N.3: FSM workflow of constraint mining. Edge label $O_k^{S_i}$ is the k -th action of state S_i ; $O_0^{S_i}$ is the default forward action. S_3 has four: accept ($O_0^{S_3}$), tighten scope ($O_1^{S_3}$), reject ($O_2^{S_3}$), fetch contradicting evidence ($O_3^{S_3}$). Double ring marks S_5 as accepting.

Table O.1: Per-case buggy detection results on Real-SE (matching the main-text *Detection Effectiveness and Precision* totals: TrainAudit 17/18, TrainCheck 5/17, Naive 0/17). Case ID uses “framework abbreviation + fixed commit short hash”; cases without a paired commit ID keep their original identifiers. Tier: A0 = PyTorch public-interface layer; A1 = framework adapter layer. ✓ = detected; × = missed. The TrainCheck column shows in parentheses *TC violations / total checks*. LC1/O-003 is the observability-boundary case (subthreshold drift; all three tools × as theoretically expected; baseline verdicts on it pending re-run).

Case ID	Framework	Tier	Triggered Constraint / Fault Pattern	TrainAudit	TrainCheck	Naive
<i>Real-SE reproducible cases</i>						
Megatron-3c637fc0d	Megatron-LM	A1	replica-cksum-equal	✓	✓ (1/316)	×
DS-d56268f3	DeepSpeed	A1	comm-dtype-matches-training	✓	×	×
DS-8cbbb53	DeepSpeed	A1	process-group-size-correct	✓	✓ (2/433)	×
DS-005afe12	DeepSpeed	A0	clip-grad-bounded	✓	✓ (11/604)	×
OLMoC-6e330ba2	OLMo-core	A0	initial-lr-present	✓	✓ (24/478)	×
Megatron-99f999a4	Megatron-LM	A1	layer-count-strict	✓	×	×
OLMo-204ad53c	OLMo	A0	checkpoint-preserve-rng	✓	×	×
OLMo-167a7ac8	OLMo	A0	data-loader-token-id-range	✓	×	×
OLMoC-2b6cf996	OLMo-core	A0	optim-step-counter-monotonic	✓	✓ (18/506)	×
Megatron-e6d56d68	Megatron-LM	A1	invocation-frequency-per-step	✓	×	×
DS-e6e8c137	DeepSpeed	A1	moe-cpu-offload grad-norm device	✓	×	×
DS-40009eb1	DeepSpeed	A1	bf16-lp-grad-clear	✓	×	×
OLMo-ce5f30d5	OLMo	A0	emb-init-std-sqrt-dmodel	✓	×	×
OLMo-a57f3803	OLMo	A0	zloss-mean-reduction	✓	×	×
OLMo-d7994c86	OLMo	—	LC1/O-003: subthreshold z-loss drift	×	×	×
OLMoC-db2b6eff	OLMo-core	A1	a100-peak-flops-2x	✓	×	×
Megatron-1505db4c	Megatron-LM	A1	chained-optimizer-param-groups-resync	✓	×	×
OLMoC-df72f4ee	OLMo-core	A0	rope-packed-doc-position-reset	✓	×	×
Total				17/18 (94.4%)	5/17 (29.4%)	0/17 (0.0%)

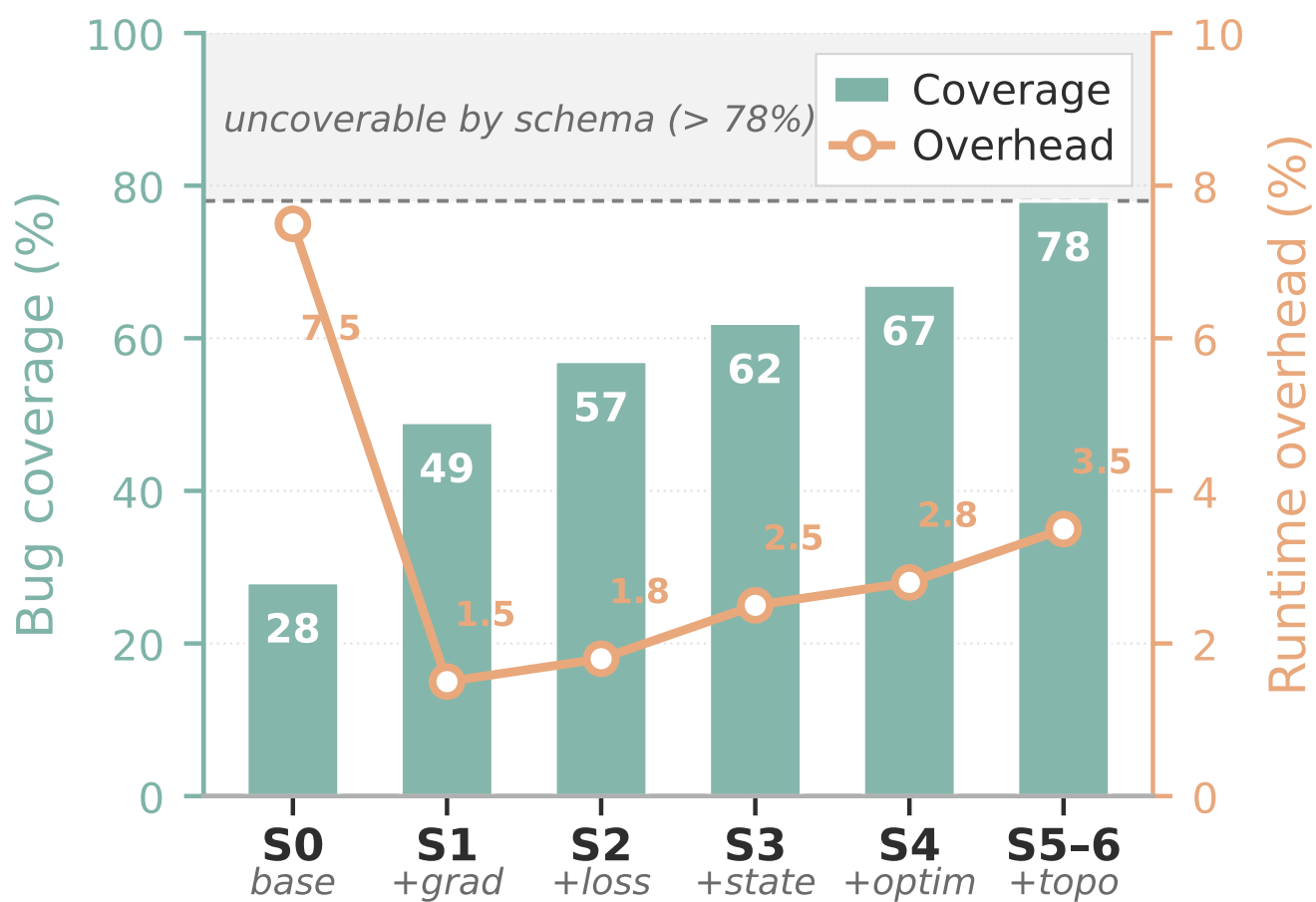


Figure O.1: Schema collection tier coverage vs. overhead. S0→S1 raises coverage 28%→49% and lowers overhead ~8%→~1.5%; S5-6 hit the 78% coverage ceiling.